

Exercise 7: Summary Comparison

1. Comparison of the Six Programs

Program	Collective(s) used	Array allocation	Manual summation on root?	Where is the result available?
sum_bcast.c	MPI_Bcast + MPI_Send / MPI_Recv	Every process allocates the full 1,000,000-element array	Yes – root receives each local sum and adds them	Root only
sum_scatter.c	MPI_Scatter + MPI_Send / MPI_Recv	Root allocates the full array; every process allocates only its chunk	Yes – root receives each local sum and adds them	Root only
sum_gather.c	MPI_Scatter + MPI_Gather	Root allocates the full array; every process allocates only its chunk; root also allocates all_sums	Yes – root manually adds the values in all_sums	Root only
sum_reduce.c	MPI_Scatter + MPI_Reduce	Root allocates the full array; every process allocates only its chunk	No – MPI_Reduce performs the summation	Root only
sum_allreduce.c	MPI_Scatter + MPI_Allreduce	Root allocates the full array; every process allocates only its chunk	No – MPI_Allreduce performs the summation	All processes
sum_scan.c	MPI_Scatter + MPI_Scan	Root allocates the full array; every process allocates only its chunk	No – MPI_Scan calculates the prefix sums	Different result for each rank

2. Execution Time Comparison

Program	2 Processes	4 Processes	8 Processes
Bcast	0.0475 s	0.1653 s	0.0896 s
Scatter	0.0324 s	0.0605 s	0.0914 s
Gather	0.0610 s	0.0448 s	0.0396 s
Reduce	0.0625 s	0.0350 s	0.0438 s
Allreduce	0.0082 s	0.0301 s	0.0393 s
Scan	0.0954 s	0.0689 s	0.0578 s

3. Fastest Approach and Why

Based on the measured execution times, MPI_Allreduce had the lowest execution time in the 2-process, 4-process, and 8-process tests among the recorded results. The MPI_Allreduce operation combines the partial sums and distributes the final result to all processes in a single collective operation. This avoids the manual communication and summation used in the earlier approaches. The exact execution time can vary depending on the system and the number of MPI processes used.

4. MPI_Scan vs MPI_Allreduce

I would choose MPI_Scan when each process needs a cumulative result that includes its own value and the values from all previous processes. MPI_Allreduce is used when every process needs the same final global result.

For example, suppose each MPI process contains a different number of records and the records need to be assigned global positions. MPI_Scan can calculate how many records belong to the previous processes. This value can be used as a global offset so that each process knows where its records should begin in the global output. Therefore, MPI_Scan is useful for global indexing, cumulative calculations, and assigning positions in parallel data processing.